SEMT30008 Week 22 Lab - Solution: LLM Agent Planning & Autonomous Workflows

Introduction

Welcome to the Autonomous AI Agents Lab!

In this 2-hour session, we will demystify the architectural and algorithmic components that turn a static LLM into a dynamic, autonomous reasoning agent capable of planning and executing multi-step tasks. Based on our lecture (Ferrag et al., 2025), we will implement the core components of agentic workflows:

1. **Foundations of Planning** (ReAct, Temperature Control)
2. **Agent Architectures** (Agentic RAG, HITL, Multi-Agent Handoffs)
3. **Protocols & Tool Use** (MCP / JSON-RPC, Tool Routing)
4. **Evaluation & Failure Modes** (Agent-as-a-Judge, Loop Prevention)

Let's dive in! Run the setup cell below to import necessary libraries.

In [ ]:
```python
# Run this first!
import json
import math
import torch
import torch.nn.functional as F
import random

print("Setup complete! Ready to build autonomous agents.")
```

Module 1: Foundations of Agentic Planning

Reference: Slides 4–11

Autonomous agents rely on a continuous loop of Perception, Memory, and Planning/ Action. The **ReAct** (Reason + Act) paradigm forces the model to think before interacting with tools. Furthermore, the **Temperature** parameter dictates whether the agent strictly exploits known rules or creatively explores new plans.

Exercise 1: The ReAct (Reason + Act) Loop

Task: Implement the core execution loop of an autonomous agent. It should continue taking actions until the planning module outputs the `"Finish"` command.

In [ ]:
```python
def execute_agent_loop(initial_task, max_iterations=5):
    state = "Thought: I need to solve this. "

    for i in range(max_iterations):
        # Simulate LLM deciding the next action
        action_type = "Search" if i < 2 else "Finish"

        # TODO 1: Check if the action_type is "Finish" to break the loop
        if action_type == ___:
            print(f"Step {i}: Task Complete!")
            ___

        print(f"Step {i}: Executing {action_type}...")
        state += f"Action {i} done. "

    return state


execute_agent_loop("Find the latest AI news.")
```

Exercise 2: Temperature Control in Planning

As discussed in the new Slide 11, Low Temperature is for strict tool-use (Exploitation), while High Temperature is for brainstorming (Exploration).

Task: Apply temperature scaling to a set of raw logits before applying softmax to see how it affects the probability of choosing alternative plans.

In [ ]:
```python
def apply_temperature(logits, temperature):
    # TODO 2: Divide the logits by the temperature
    scaled_logits = logits / ___

    # TODO 3: Apply torch.softmax across the last dimension (dim=-1)
    probabilities = torch.softmax(___, dim=-1)
    return probabilities

plan_logits = torch.tensor([5.0, 2.0, 1.0]) # Plan A is clearly favored
print(f"Strict Planning (T=0.1): {apply_temperature(plan_logits, 0.1).tolist()
print(f"Creative Brainstorming (T=2.0): {apply_temperature(plan_logits, 2.0).t
```

Module 2: Agent Architectures & Frameworks

Reference: Slides 6, 16, 19

Modern systems move past single agents. We use **Agentic RAG** to decide *if* search is needed, **Human-in-the-Loop (HITL)** for safety, and **Swarm-style** handoffs for Multi-Agent Systems.

Exercise 3: Agentic RAG Router

Unlike traditional RAG which *always* searches, Agentic RAG evaluates its internal confidence first.

Task: Write a router that outputs `"Direct Answer"` if confidence is high, and `"Search Vector DB"` if confidence is low.

In [ ]:
```python
def agentic_rag_router(query, internal_confidence, threshold=0.8):
    # TODO 4: Return "Direct Answer" if internal_confidence is greater than or
    if internal_confidence >= ___:
        return "Direct Answer"
    else:
        # TODO 5: Otherwise, return "Search Vector DB"
        return ___

print(f"Query: 'What is 2+2?' -> {agentic_rag_router('What is 2+2?', 0.99)}")
print(f"Query: 'Who won the 2028 Olympics?' -> {agentic_rag_router('Who won th
```

Exercise 4: Human-in-the-Loop (User Confirmation)

To prevent agents from making dangerous autonomous decisions (e.g., deleting a database), we implement a HITL gatekeeper.

Task: Pause the workflow if the action is deemed "critical" (like `"delete"` or `"write"`).

In [ ]:
```python
def hitl_gatekeeper(action_intent):
    critical_actions = ["delete_file", "write_database", "send_email"]

    # TODO 6: Check if action_intent is in the critical_actions list
    if action_intent in ___:
        return "PAUSED: Waiting for Human User Confirmation (ROC)."

    return "Executing autonomously."

print(hitl_gatekeeper("read_weather_api"))
print(hitl_gatekeeper("delete_file"))
```

Exercise 5: Multi-Agent Handoff (Swarm Framework)

In OpenAI's Swarm architecture, agents don't use complex centralized orchestrators; they simply return the *next agent* to be called based on context.

Task: Route the user to the `CodeAgent` if the query contains "python", otherwise route to the `GeneralAgent`.

In [ ]:
```python
def swarm_handoff(user_query):
```

```
    # TODO 7: Check if "python" is in the user_query (case-insensitive)
    if "python" in user_query.___():
        return "Transferring to CodeAgent..."

    # TODO 8: Default return for other queries
    return ___

print(swarm_handoff("Write a python script to scrape data."))
print(swarm_handoff("Translate hello to French."))
```

Module 3: Protocols & Tool Use

Reference: Slides 56–60

Agents communicate with tools and each other using strict protocols like Anthropic's **Model Context Protocol (MCP)** and Google's **A2A**, which rely on JSON-RPC 2.0.

Exercise 6: Formatting an MCP / JSON-RPC Request

Task: Construct a valid JSON-RPC 2.0 message for an agent invoking a tool via MCP.

In [ ]:
```
def create_mcp_request(call_id, method_name, params_dict):
    # TODO 9: Construct the JSON-RPC payload matching the MCP spec
    payload = {
        "jsonrpc": "2.0",
        "id": ___,
        "method": ___,
        "params": ___
    }
    return json.dumps(payload, indent=2)

print(create_mcp_request(101, "read_local_file", {"filepath": "/docs/plan.txt"
```

Exercise 7: Dynamic Tool Parsing

LLMs output text. To execute a tool, we must parse the tool name and arguments from the LLM's generated string.

Task: Extract the tool and arguments from a simulated JSON string generated by the LLM.

In [ ]:
```
def parse_llm_tool_call(llm_output_string):
    try:
        # TODO 10: Parse the JSON string into a Python dictionary
        data = json.___(___)
        return data["tool"], data["args"]
    except Exception as e:
```

```
        return "Error: Invalid JSON format generated by agent."

llm_response = '{"tool": "calculator", "args": {"equation": "15 * 4"}}'
tool, args = parse_llm_tool_call(llm_response)
print(f"Invoking {tool} with arguments {args}")
```

Module 4: Evaluation & Failure Modes

Reference: Slides 34, 64

Multi-agent systems often fail due to infinite loops, ignoring instructions, or poor verification. We combat this using strict termination conditions and "Agent-as-a-Judge" grading.

Exercise 8: Preventing Infinite Reasoning Loops

A common failure mode in MAS is agents talking back and forth endlessly without resolving the task.

Task: Implement a hard token/step limit that raises an exception if the agent plans for too long.

In [ ]:
```
def safe_planning_loop(max_steps=10):
    step_count = 0
    while True:
        # Simulate planning step
        step_count += 1

        # TODO 11: Check if step_count exceeds max_steps
        if ___ > ___:
            # TODO 12: Raise a RuntimeError to stop the infinite loop
            raise ___("Agent Failure: Maximum planning steps exceeded. Termina

        if step_count == 15: # Agent naturally finishes at step 15 (but will b
            return "Success"

try:
    safe_planning_loop()
except Exception as e:
    print(e)
```

Exercise 9: Process Reward for Math Planning (ProcessBench)

When evaluating agent reasoning (e.g., in `ProcessBench`), we want to penalize the *exact step* where the logic failed, rather than just grading the final answer.

Task: Iterate through reasoning steps. Give +1 reward for valid steps, and -1 for the first invalid step (and stop scoring).

In [ ]:
```python
def process_reward_model(steps, error_index):
    rewards = []
    # TODO 13: Loop through the index and content of 'steps' using enumerate()
    for i, step in ___(steps):
        if i == error_index:
            # TODO 14: Append -1 for the error step and break the loop
            rewards.append(___)
            ___
        else:
            rewards.append(1)

    return rewards

plan = ["x = 5", "y = 10", "x + y = 100", "return 100"]
error_idx = 2 # The math is wrong at step 2
print(f"Step Rewards: {process_reward_model(plan, error_idx)}")
```

Exercise 10: Agent-as-a-Judge

Instead of humans grading code, we use an LLM Agent to judge another Agent's output based on alignment and correctness (saves 97% of evaluation costs).

In [ ]:
```python
def agent_judge_score(correctness_score, format_score):
    # TODO 15: Multiply correctness by 0.60 and formatting by 0.40
    final_score = (correctness_score * ___) + (format_score * ___)
    return final_score

# The coding agent got the math right (1.0), but output XML instead of JSON (0
print(f"Final Agent-as-a-Judge Score: {agent_judge_score(1.0, 0.2):.2f}")
```